



UNIVERSIDAD TÉCNICA DE AMBATO
FORMULARIO DE GUÍAS PRÁCTICAS DE APLICACIÓN Y
EXPERIMENTACIÓN DE LOS APRENDIZAJES



FACULTAD: INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL		CARRERA: Software
NIVEL: PRIMERO	UOC: Unidad Básica	ASIGNATURA: ALGORITMOS Y LOGICA DE PROGRAMACION
CICLO ACADÉMICO: JULIO 2026 - DICIEMBRE 2026		
NOMBRE DEL ESTUDIANTE: Matías Alejandro Pico Cepeda		

Lenguajes de Programación

LA HISTORIA Y EVOLUCIÓN

1. Introducción y Fundamentos

2. Los Antecedentes y la Era del Hardware Primitivo (Década de 1940)

3. La Irrupción de los Lenguajes de Alto Nivel (Décadas de 1950 - 1960)

4. La Consolidación Estructurada, la Orientación a Objetos y la Web (Décadas de 1970 - 1990)

5. El Panorama Tecnológico Actual

6. Conclusión

7. Infografía

Objetivo: Analizar la evolución cronológica de los lenguajes de programación, desde sus antecesores de bajo nivel hasta las tecnologías actuales, con el fin de identificar las bases que facilitan la toma de decisiones en la creación de proyectos de software.

1. Introducción y fundamentos

Los lenguajes de programación son sistemas formales de notación diseñados para describir algoritmos y transmitir instrucciones comprensibles tanto para los seres humanos como para las máquinas. Su evolución a lo largo de las décadas no ha sido lineal, sino que responde directamente a la Ley de Moore, al abaratamiento del hardware, al incremento de la potencia de procesamiento y, sobre todo, a la necesidad de gestionar proyectos de software cada vez más masivos y complejos. Estudiando sus orígenes y antecesores nos permite comprender la lógica subyacente de la computación moderna, facilitando la toma de decisiones técnicas al momento de desarrollar soluciones de software adaptadas a requerimientos reales.

2. Los Antecedentes y la Era del Hardware Primitivo (Década de 1940)

En los albores de la computación electrónica, las computadoras no se "programaban" en el sentido moderno, sino que se reconfiguraban físicamente mediante cables, interruptores y paneles de conexiones (como en la ENIAC). Posteriormente, con la arquitectura de programas almacenados, surgieron las primeras formas de software rudimentario:

- **El Código Máquina (Primera Generación):** Representa el nivel más bajo de abstracción. Consiste estrictamente en secuencias binarias (0\$ y 1\$) que el procesador interpreta directamente para activar circuitos internos. Si bien era el único lenguaje nativo de la máquina, su escritura era extremadamente tediosa, propensa a errores humanos catastróficos y totalmente dependiente de la arquitectura específica del procesador utilizado.
- **El Lenguaje Ensamblador (Assembly):** Para mitigar la complejidad del código binario, a finales de los años 40 se introdujo el ensamblador. Este utilizaba abreviaturas nemotécnicas (códigos mnemónicos como `ADD`, `MOV`, `SUB`, `JMP`) que guardaban una correspondencia directa de uno a uno con las instrucciones de la máquina. Aunque supuso un gran avance en legibilidad, seguía estando profundamente ligado al hardware, requiriendo un conocimiento exhaustivo de los registros y la memoria física del equipo.

3. La Irrupción de los Lenguajes de Alto Nivel (Décadas de 1950 - 1960)

Con el crecimiento de la industria y la necesidad de aplicar la computación a la ciencia y los negocios, la dependencia del ensamblador se volvió insostenible. Así nació la segunda y tercera generación de lenguajes, caracterizados por utilizar una sintaxis cercana al idioma inglés y a la notación matemática:

- **FORTRAN (1957):** Desarrollado por un equipo de IBM liderado por John Backus, su nombre proviene de *Formula Translation*. Es considerado el primer lenguaje de alto nivel comercialmente exitoso de la historia. Estaba optimizado para cálculos numéricos complejos, ingeniería y aplicaciones científicas, sentando las bases de la compilación de código.
- **LISP (1958):** Creado por John McCarthy, introdujo conceptos revolucionarios procedentes del cálculo lambda, convirtiéndose en el lenguaje pionero e indiscutible para la investigación en Inteligencia Artificial y el procesamiento de listas simbólicas.
- **COBOL (1959):** Impulsado por el Departamento de Defensa de los Estados Unidos (a través del comité CODASYL) y liderado por figuras como Grace Hopper, su objetivo fue estandarizar un lenguaje común para la gestión de datos comerciales, administrativos y financieros. Su legibilidad basada en estructuras similares al inglés comercial hizo que perdurara en instituciones bancarias y gubernamentales durante décadas.
- **BASIC (1964):** Diseñado por John Kemeny y Thomas Kurtz con un propósito eminentemente educativo, democratizó el acceso a la programación para estudiantes que no pertenecían estrictamente al ámbito científico o matemático.

4. La Consolidación Estructurada y la Orientación a Objetos (Décadas de 1970 - 1990)

A medida que los programas aumentaban en líneas de código, se hizo evidente el problema del "código espagueti" (saltos descontrolados mediante instrucciones como GOTO), lo que impulsó la programación estructurada y, posteriormente, la modularidad:

C (1972): Creado por Dennis Ritchie en los Laboratorios Bell de AT&T como una evolución de los lenguajes BCPL y B. Diseñado originalmente para reescribir el sistema operativo UNIX, el lenguaje C combinó las capacidades de bajo nivel (manipulación de memoria mediante punteros) con la estructura de alto nivel. Se convirtió en el antecesor directo de la gran mayoría de lenguajes modernos (C++, C#, Java, JavaScript).



C++ (1983): Desarrollado por Bjarne Stroustrup como una extensión del lenguaje C, introdujo formalmente el paradigma de la Programación Orientada a Objetos (POO), incorporando conceptos fundamentales como clases, herencia, polimorfismo y encapsulamiento, permitiendo modelar sistemas complejos del mundo real.



Python (1991): Creado por Guido van Rossum con una filosofía enfocada en la legibilidad del código y una sintaxis limpia y minimalista, reduciendo drásticamente el costo de mantenimiento del software y facilitando el autoaprendizaje.



Java (1995): Desarrollado por James Gosling en Sun Microsystems bajo el lema "Write Once, Run Anywhere" (Escribe una vez, ejecuta en cualquier lugar) gracias a la Máquina Virtual de Java (JVM), convirtiéndose en el estándar de la industria para aplicaciones empresariales, de servidor y dispositivos móviles (Android).



JavaScript (1995): Creado por Brendan Eich en Netscape, nació originalmente para dotar de dinamismo e interactividad a las páginas web del lado del cliente en los navegadores. Con el tiempo, evolucionó drásticamente hasta convertirse en un lenguaje multiparadigma (soporta programación imperativa, funcional y orientada a objetos) y expandirse masivamente al lado del servidor (backend) mediante entornos como Node.js.



5. El Panorama Tecnológico Actual

En la actualidad, el ecosistema de desarrollo de software es sumamente heterogéneo y dinámico, donde la industria promueve la especialización y la interoperabilidad:

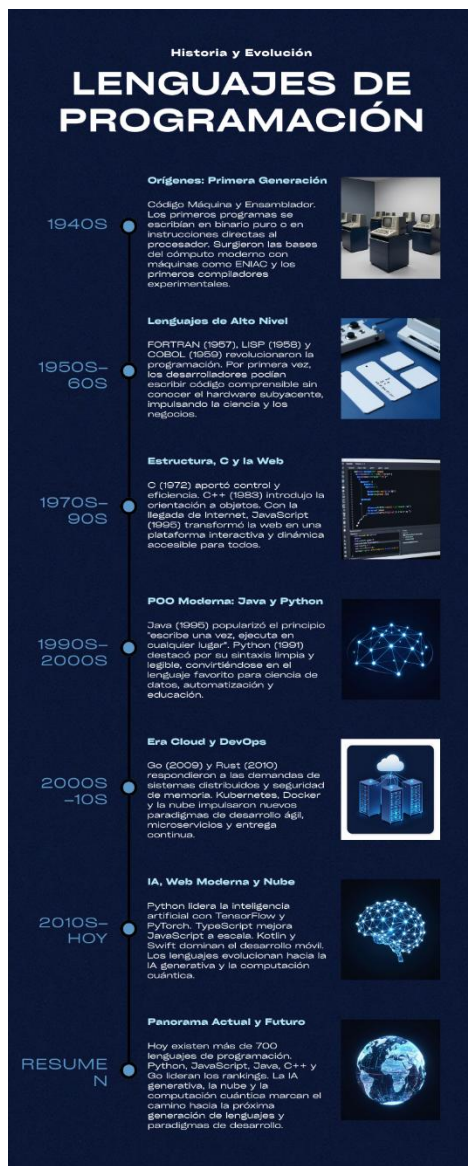
- **Desarrollo Web Full-Stack: JavaScript** y su variante tipada **TypeScript** dominan de manera absoluta tanto el desarrollo del lado del cliente como del servidor.
- **Inteligencia Artificial y Ciencia de Datos: Python** se ha consolidado como el estándar mundial gracias a su vasto ecosistema de librerías especializadas.
- **Sistemas de Alto Rendimiento y Cloud Computing:** Lenguajes modernos como **Rust** y **Go** lideran las preferencias para infraestructuras críticas y distribuidas por su seguridad de memoria y concurrencia.

6. Conclusión

El conocimiento de los lenguajes de programación y de sus antecesores no es meramente anecdótico; dota al desarrollador de una visión panorámica de la evolución tecnológica. Esto es fundamental para entender la resolución de problemas algorítmicos complejos y permite elegir el lenguaje que mejor se adapte a las necesidades del cliente en proyectos de software.

7. Infografía

<https://canva.link/0t5ij856a5dtasc>



I. TEMA: SW-AyLP-APE-01: Lenguajes de Programación
II. OBJETIVO: Conocer los lenguajes de programación de la actualidad y sus antecesores.
III. MODALIDAD: Presencial
IV. TIEMPO DE DURACIÓN: Presenciales: 2 No Presenciales: 0
V. INSTRUCCIONES: - El trabajo se desarrollará de manera individual. - Realice una consulta sobre LA HISTORIA Y EVOLUCIÓN DE LOS LENGUAJES DE PROGRAMACIÓN. - Seleccionar una herramienta que permita realizar infografías. - Realizar un infografía a manera de resumen de la lectura realizada. - Subir el documento al aula virtual en el formato de Guías APE en PDF.
VI. LISTADO DE EQUIPOS, MATERIALES Y RECURSOS: Inteligencia artificial, TAC Inteligencia artificial, TAC - Inteligencia artificial, TAC - Computador o laptop. - Plataforma educativa de la UTA. - Internet para consultas (fuentes confiables).
VII. ACTIVIDADES POR DESARROLLAR: Desarrollar la guía APE en base al formato y las instrucciones brindadas
VIII. RESULTADOS OBTENIDOS: Conoce los lenguajes de programación de la actualidad y sus antecesores.
IX. HABILIDADES BLANDAS:
X. CONCLUSIONES: El conocimiento de los lenguajes de programación permite elegir el que mejor se adapte a las necesidades del cliente en proyectos de software.
XI. RECOMENDACIONES: - Si se manifiestan dudas sobre el tema de la APE, se invita a los estudiantes acudir a las tutorías académicas.



UNIVERSIDAD TÉCNICA DE AMBATO
FORMULARIO DE GUÍAS PRÁCTICAS DE APLICACIÓN Y EXPERIMENTACIÓN DE LOS
APRENDIZAJES



BIBLIOGRAFÍA

Autores	Año	Título	Nro Edición	Editorial	Pais/Ciudad	Nro Ejemplares	Nro Páginas	Tipo
Contreras Castro, Mario Dustano	2023	Lógica de programación a partir del pensamiento lógico con software DFD, PSeInt y LPP	1	Ediciones USTA	España/Valencia	40	90	Virtual
Código/Ubicación base de datos:								
Comentario: Obra de consulta obligatoria para desarrollar pensamiento lógico, pseudocódigo, diagramas de flujo y estructuras de control; se articula con las unidades 1 y 2.								
URL: https://0x102prpx-y-https-elibro-net.itmsp.museknowledge.com/es/lc/uta/titulos/288308								

Autores	Año	Título	Nro Edición	Editorial	Pais/Ciudad	Nro Ejemplares	Nro Páginas	Tipo
Barquilla Galeano, María Isabel	2025	Java programación: teoría y ejemplos	1	RA-MA Editorial	España/Madrid	40	578	Virtual
Código/Ubicación base de datos:								
Comentario: Texto de consulta obligatoria para tipos de datos, estructuras de control, arreglos, métodos, programación orientada a objetos y archivos; se articula con las unidades 2, 3								
URL: https://0x102prtt-y-https-elibro-net.itmsp.museknowledge.com/es/lc/uta/titulos/285282								

Autores	Año	Título	Nro Edición	Editorial	Pais/Ciudad	Nro Ejemplares	Nro Páginas	Tipo
Dueñas Meza José María, Valdés Benjumea Jaime José, Rodríguez Niño María Consuelo	2020	Didáctica para el aprendizaje de algoritmos	1	Sello Editorial UNAD	Colombia/Bogota	40	150	Virtual
Código/Ubicación base de datos:								
Comentario: Este libro contiene conceptos básicos sobre el diseño de algoritmos explicados de una manera clara y didáctica.								
URL: https://0x102pruy-y-https-elibro-net.itmsp.museknowledge.com/es/lc/uta/titulos/288503								

Autores	Año	Título	Nro Edición	Editorial	Pais/Ciudad	Nro Ejemplares	Nro Páginas	Tipo
Blasco, Francisco	2019	Programación orientada a objetos en Java	1	Ediciones de la U	Colombia/Bogota	40	589	Virtual



UNIVERSIDAD TÉCNICA DE AMBATO
FORMULARIO DE GUÍAS PRÁCTICAS DE APLICACIÓN Y EXPERIMENTACIÓN DE LOS
APRENDIZAJES



Código/Ubicación base de datos:	
Comentario:	El objetivo principal de este libro es presentar al lector los fundamentos de la Programación Orientada a Objetos.
URL:	https://0x102pry9-y-https-elibro-net.itmsp.museknowledge.com/es/lc/uta/titulos/127125

Autores	Año	Título	Nro Edición	Editorial	Pais/Ciudad	Nro Ejemplares	Nro Páginas	Tipo
Barquilla Galeano, María Isabel	2025	Java programación: teoría y ejemplos	1	RA-MA	España/Madrid	20	578	Virtual
Código/Ubicación base de datos:								
Comentario:	Esta obra es una guía completa y progresiva para aprender a programar con uno de los lenguajes más utilizados del mundo. Dirigido tanto a estudiantes como a profesionales que deseen consolidar sus conocimientos.							
URL:	https://0x1025hkn-y-https-elibro-net.itmsp.museknowledge.com/es/lc/uta/titulos/285282							



UNIVERSIDAD TÉCNICA DE AMBATO
FORMULARIO DE GUÍAS PRÁCTICAS DE APLICACIÓN Y
EXPERIMENTACIÓN DE LOS APRENDIZAJES



VALIDACIÓN DE LAS GUÍAS DE PRÁCTICAS

Fecha de elaboración

DOCENTE PLANIFICADOR UTA

Ing. M.Sc. MARCOS RAPHAEL BENITEZ ALDAS
JOSE RUBEN CAIZA CAIZABUANO

Coordinador Unidad de Organización Curricular

PhD VICTOR FILIBERTO PEÑAFIEL GAIBOR

Coordinador de Carrera

Ing. Mg. MARCO VINICIO GUACHIMBOZA VILLALVA